

Compiladores PIC16 desde dentro

Guía práctica de desarrollo por fases con pic16cc

Prólogo

Este libro está escrito para desarrolladores que quieren entender un compilador por dentro y, al mismo tiempo, aprender a construir uno propio siguiendo un recorrido lógico.

El caso de estudio es pic16cc, el compilador Rust de este repositorio para microcontroladores PIC16 clásicos de 14 bits. La idea no es presentar una lista de archivos, sino un mapa mental: qué problema resuelve cada fase, por qué se introdujo en ese orden y cómo se conecta con el resto del sistema.

La documentación base de esta guía es el propio repositorio, especialmente README.md, DESIGN.md y docs/ir/overview.md, junto con las notas por fase en docs/ir/.

Estado actual del proyecto estudiado:

- binario de compilación: picc
 - binario opcional de simulación: pic16-sim
 - dispositivos soportados: PIC16F628A y PIC16F877A
 - salida principal: Intel HEX programable
 - salidas auxiliares: .map, .lst y volcados opcionales de AST, IR y ASM
 - estado funcional actual: Phase 46
-

1. Qué es un compilador

Un compilador es un programa que transforma un lenguaje fuente, escrito por humanos, en una representación que una máquina concreta puede ejecutar.

En este proyecto la entrada es un subconjunto de C y la salida es Intel HEX para PIC16.

La idea importante no es sólo traducir texto. Un compilador también debe:

- entender la sintaxis del lenguaje
- validar reglas semánticas
- decidir tipos y conversiones
- resolver memoria, llamadas y retorno
- generar código correcto para la arquitectura destino

Un ejemplo mínimo es este:

```
PORTB = 1;
```

La traducción conceptual no es “cambiar texto”. Es algo como:

1. reconocer tokens
2. formar una asignación
3. comprobar que PORTB es un destino válido
4. producir una carga de literal
5. emitir una escritura al SFR
6. codificarlo en palabras PIC16
7. escribirlo en Intel HEX

2. Las fases de cualquier compilador

Todo compilador serio tiende a separar el trabajo en fases. No es una convención académica vacía; es una forma de mantener cada problema en su sitio.

En pic16cc el pipeline general es este:

1. preprocesado
2. lexing
3. parsing
4. análisis semántico
5. lowering a IR tipada
6. optimización IR
7. backend PIC16 compartido
8. codificación de instrucciones de 14 bits
9. emisión Intel HEX

La utilidad de esta separación se entiende con un ejemplo sencillo:

```
x = a + b;
```

La misma línea significa cosas distintas según la fase:

- como texto, es sólo una secuencia de caracteres
- como tokens, son piezas léxicas
- como AST, es una asignación con una suma
- como semántica, es una asignación válida entre tipos concretos
- como IR, puede ser una copia de un resultado temporal
- como backend, puede convertirse en cargas, suma y store

Un error frecuente al empezar a escribir compiladores es intentar resolver todo demasiado pronto. La separación por fases evita eso.

3. Frontend y backend

La división más importante en un compilador es la separación entre frontend y backend.

El frontend se ocupa de entender el programa fuente. El backend se ocupa de bajarlo a la máquina destino.

En pic16cc el frontend vive principalmente en src/frontend/ y el backend PIC16 compartido en src/backend/pic16/midrange14/.

Frontend

El frontend incluye:

- preprocesador
- lexer

- parser
- análisis semántico

Su misión es convertir texto en un programa tipado y bien formado.

Backend

El backend se ocupa de:

- elegir cómo representar la operación en PIC16
- gestionar banking y paging
- aplicar el ABI
- usar helpers runtime cuando haga falta
- producir ensamblador, codificación y HEX

La diferencia práctica es ésta:

- el frontend sabe que existe $x + 1$
- el backend sabe cómo convertir eso en instrucciones PIC16 reales

Esa separación es una de las ideas más importantes de todo el libro.

4. El PIC16 como caso de estudio

PIC16 es un destino excelente para aprender compiladores porque obliga a hacer explícitas muchas decisiones que en arquitecturas más cómodas se esconden.

Qué complica a PIC16

PIC16 introduce problemas muy concretos:

- muy pocos registros generales
- acumulador principal W
- RAM bancarizada
- memoria de programa paginada
- stack hardware limitado a retornos
- aritmética de 8 bits sobre valores de 16 o 32 bits
- acceso indirecto con FSR e INDF

Eso obliga al compilador a ser muy honesto.

Por qué esto es útil para aprender

Porque el proyecto no puede “inventarse” abstracciones que la máquina no tiene. Cada decisión importante se ve con claridad:

- cómo se pasan los argumentos
- dónde viven los locales
- cuándo se cambia de banco
- cuándo se cambia de página
- cuándo se usa un helper runtime

Un ejemplo simple

```
unsigned char add1(unsigned char x) {  
    return x + 1;  
}  
  
void main(void) {  
    TRISB = 0x00;  
    PORTB = add1(3);  
}
```

Este ejemplo ya fuerza varias decisiones:

- llamada a función
- retorno en W
- acceso a SFR
- uso de un valor literal

5. El primer paso lógico

Si tu objetivo es construir un compilador, el primer paso lógico no es soportar “mucho lenguaje”. Es cerrar una cadena mínima de extremo a extremo.

Eso significa poder tomar una entrada pequeña y producir una salida válida, aunque el lenguaje soportado sea muy limitado.

En pic16cc ese punto de partida se construyó alrededor de esta idea:

1. leer el fichero fuente
2. expandir includes y macros básicos
3. tokenizar
4. parsear
5. validar semántica
6. bajar a IR
7. generar PIC16
8. codificar
9. emitir HEX

La lección es simple: antes de añadir features, hay que tener una tubería fiable. Un buen programa inicial es uno que muestre varias capas a la vez sin ser demasiado grande:

```
PORTB = 1;
```

Ese ejemplo parece trivial, pero obliga a resolver todo el recorrido básico del compilador.

6. Fases 2 a 7: del núcleo mínimo al primer compilador útil

Estas fases construyen la base del compilador. Son las que convierten un experimento en una herramienta que ya se puede estudiar con seriedad.

Phase 2: enteros de 16 bits y comparaciones

La primera ampliación importante fue salir del mundo de 8 bits puros y tratar int y unsigned int como valores de 16 bits.

Esto importa porque PIC16 es una máquina de 8 bits, pero el lenguaje necesita comparaciones y valores más anchos.

La idea clave es que el compilador debe saber cómo representar un entero de 16 bits sin pedir ayuda al hardware que no existe.

Ejemplo conceptual:

```
0x1234 -> byte bajo 0x34, byte alto 0x12
```

Phase 3: arrays, punteros y modelo de memoria

Aquí aparece uno de los temas centrales de cualquier compilador: memoria.

La fase introduce:

- arrays unidimensionales
- punteros a datos
- &x y *ptr
- AddrOf, LoadIndirect, StoreIndirect

El backend empieza a usar FSR e INDF para navegar direcciones dinámicas.

Ejemplo:

```
*ptr = value;
```

Conceptualmente eso ya no es una simple asignación. Es materializar una dirección, apuntarla y escribir a través de ella.

Phase 4: stack-first ABI y frames

Este es uno de los grandes hitos del proyecto.

Antes de esta fase, el compilador no tenía una historia sólida para:

- 3 o más argumentos
- llamadas anidadas
- locales por invocación
- temporales por función

La solución fue un ABI stack-first con pila de software.

Conceptos clave:

- ABI: contrato binario entre caller y callee
- frame: bloque de stack perteneciente a una invocación
- SP: stack pointer
- FP: frame pointer

Ejemplo conceptual de frame:

```
args | saved FP | locals | temps
```

Esto convierte las llamadas en algo robusto y explicable.

Phase 5: helpers aritméticos

PIC16 no tiene multiplicación, división ni módulo hardware cómodos para C.

La solución fue introducir helpers runtime como:

- __rt_mul_*
- __rt_div_*
- __rt_mod_*
- __rt_sh*

Eso permite compilar operadores como *, /, %, << y >> sin inventar atajos frágiles.

Ejemplo:

```
unsigned int r = a * b;
```

La multiplicación puede bajar a un algoritmo shift-and-add o a rutas inline simples si el caso lo permite.

Phase 6: ISR

La interrupción introduce una ruta de ejecución especial.

El compilador debe emitir:

- vector de interrupción en 0x0004
- guardado conservador de contexto
- retorno con retfie

La semántica también restringe el cuerpo de la ISR para evitar corrupción de estado.

Phase 7: optimización conservadora

Una vez que el compilador ya genera código correcto, llega el momento de hacerlo menos torpe.

Phase 7 introduce:

- folding de constantes
- propagación de constantes
- eliminación de código muerto
- compactación de temporales
- peephole local

La lección es importante: optimizar no significa arriesgar la corrección. Significa reducir trabajo innecesario sin romper contratos.

7. Fases 8 a 18: crecer sin perder el control

Estas fases amplían el lenguaje y fortalecen el modelo interno sin abandonar la misma filosofía: soportar cosas reales, pero de forma coherente.

Phase 8: typedef, enum y struct

La fase añade tipos agregados y alias de tipos.

Esto es importante porque sin tipos compuestos el lenguaje sigue siendo demasiado estrecho para firmware serio.

Ejemplo:

```
typedef unsigned char byte;

struct Pair {
    byte a;
    byte b;
};
```

Phase 9: switch

La fase introduce switch, case, default, break y fallthrough controlado. El backend evita jump tables y usa cadenas de comparación, que son más fáciles de verificar en PIC16.

Phase 10: string literals e inicialización estática

La fase añade:

- literales de cadena
- arrays inicializados por cadena
- const estático respaldado por RAM cuando el modelo lo exige
- inicialización en startup

Esto hace visible una idea crucial: en este proyecto, la inicialización no es magia del runtime, sino código explícito de arranque.

Phase 11: agregados anidados

La fase mejora struct y arrays dentro de structs.

También añade inicialización anidada, zero-fill y asignación estructural por bytes.

Phase 12: punteros más ricos

Se amplía el modelo con:

- punteros a punteros dentro del modelo permitido
- const y combinaciones de qualifiers
- comparación y resta de punteros compatibles
- inicialización de strings en RAM

Phase 13 y 14: ROM y lecturas ROM

Aquí aparece una separación muy importante: no todo vive en RAM.

El compilador introduce __rom, tablas RETLW y lecturas explícitas con __rom_read8 y __rom_read16.

Esto es esencial para recursos pequeños, porque permite almacenar datos fijos en memoria de programa.

Phase 15 y 16: unions, bitfields y arrays multidimensionales

Estas fases amplían el soporte de agregados:

- union
- inicializadores de union
- bitfields sin signo
- arrays multidimensionales en RAM
- indexación encadenada

Conceptualmente, el compilador empieza a dominar la composición de layouts más complejos.

Phase 17: function pointers

Se introduce una forma controlada de puntero a función.

No se modelan llamadas PIC16 arbitrarias con punteros crudos; se usan trampolines y dispatch IDs.

Eso evita introducir una abstracción que el hardware no puede sostener de forma segura.

Phase 18: análisis de llamadas y stack

La fase añade control más fino del stack y del grafo de llamadas.

Conceptos clave:

- expansión del call graph
- estimación de stack
- detección de recursion
- restricciones de overflow

Es una fase de madurez. Ya no basta con compilar; hay que saber si el programa cabe y si la pila es segura.

8. Fases 19 a 26: validación, 32 bits, fixed-point y configuración de dispositivo

Estas fases consolidan el compilador como herramienta usable para firmware real.

Phase 19 y 20: simulación y ejecución validada

Se añade un simulador interno y una CLI de depuración con pic16-sim.

Esto permite ejecutar programas compilados y comprobar resultados sobre RAM y SFR.

La lección es clara: un compilador serio necesita una forma de verificar lo que emite.

Phase 21: long y unsigned long

El proyecto pasa a soportar enteros de 32 bits con ABI y helpers adecuados.

Esto obliga a tratar de forma explícita:

- almacenamiento little-endian de 4 bytes
- retornos de 32 bits
- operaciones de suma, resta, comparación y shifts

Phase 22, 23 y 24: fixed-point

La familia fixed-point aparece en varias fases:

- tipos fijos con escalas conocidas
- literales decimales fijos
- folding de constantes
- multiplicación y división dinámica con helpers

Esto es muy instructivo porque muestra cómo introducir un sistema numérico nuevo sin mezclarlo con float todavía.

Phase 25: recursos y límites del target

El compilador empieza a validar si el programa cabe realmente en el dispositivo seleccionado.

Se reportan:

- uso de memoria de programa
- uso de RAM

- stack estimado
- contribución de helpers
- tablas ROM

Phase 26: configuración y validación HEX

Se añaden:

- `#pragma config`
- `__config(0x....)` como fallback
- validación final de HEX
- flujo de programación externo

Aquí aparece una verdad importante para todo compilador embebido: no basta con generar binario. Ese binario tiene que ser válido para el dispositivo real.

9. Fases 27 a 35: float, paging, catálogo de helpers y compacidad

Estas fases llevan el compilador a una zona más delicada: precisión numérica, selección de helpers y seguridad de página.

Phase 27 a 30: float finito y tablas ROM

El proyecto introduce float como valor de 32 bits y soporta:

- literales decimales
- almacenamiento en RAM
- conversiones con enteros y fixed-point
- comparaciones dinámicas
- tablas ROM de float

La idea central es que el compilador trabaja con un modelo finito y controlado, no con todas las peculiaridades IEEE completas.

Phase 31 y 32: seguridad de página y layout

PIC16 tiene páginas de programa. Si el compilador salta o llama sin preparar bien la página, el programa falla.

Estas fases introducen:

- emisión segura de goto y call
- validación final de edges
- metadata de página en mapas y listados
- relajación de setpage redundantes

Esta parte es muy importante porque enseña que el backend no sólo genera instrucciones: también valida invariantes de layout.

Phase 33 y 34: catálogo y compactación de helpers

El proyecto centraliza metadata de helpers runtime:

- categoría
- tamaño ABI
- dependencias
- coste estimado

- restricciones de target

Después, `--runtime-profile small` empieza a seleccionar variantes más compactas.

Esto es útil porque muestra una evolución típica de compilador real: primero soportar algo, luego medirlo, luego compactarlo sin perder claridad.

Phase 35: compacidad de helpers fijos y flotantes

La lógica de compactación se extiende a operaciones fixed-point y a ciertas rutas de float.

La idea no es esconder la complejidad, sino controlar el tamaño y el coste de runtime.

10. Fases 36 a 46: math.h finito, sqrtf, min/max y trigonometría

Estas fases convierten el soporte numérico en una biblioteca pequeña pero útil.

Phase 36: math.h mínimo

Se introduce un subconjunto pequeño de funciones como:

- `fabsf`
- `truncf`
- `floorf`
- `ceilf`
- `roundf`
- `sqrtf`
- `fminf`
- `fmaxf`
- `sinf`
- `cosf`

La lección es importante: una biblioteca matemática para un microcontrolador no tiene por qué parecerse a la de un sistema operativo grande.

Phase 37, 38 y 39: sqrtf y perfiles de precisión

`sqrtf` empieza como helper compacto y luego se vuelve sensible al perfil matemático.

Conceptos clave:

- `compact`
- `balanced`
- `precise`
- `folding constante`

Esto enseña que una operación puede tener varias implementaciones válidas según el coste y la precisión deseada.

Phase 40: harness de precisión numérica

El proyecto añade validación contra resultados esperados y tolerancias documentadas.

Esto es una lección práctica muy fuerte: cuando el compilador ya tiene operaciones numéricas más sofisticadas, hay que medirlas, no sólo asumir que funcionan.

Phase 41 y 42: fminf y fmaxf

Se introducen funciones de selección entre dos float finitos y luego se compactan mediante un comparador común.

La idea importante es la reutilización: una comparación buena puede servir para varias operaciones de alto nivel.

Phase 43, 44 y 45: sinf, cosf y shared core

La trigonometría llega con una estrategia clara:

- tablas ROM
- un núcleo compartido
- wrappers para sinf y cosf
- validación por simulador

Esto evita duplicar lógica y reduce el coste de runtime.

Phase 46: reducción de rango moderada

La fase más reciente mejora la evaluación trigonométrica para rangos más amplios sin abandonar el modelo finito y controlado.

Esta fase es muy representativa del estilo del proyecto:

- no promete lo imposible
- documenta lo que hace
- valida lo que emite
- mantiene el control del tamaño y la precisión

11. Qué enseña pic16cc a quien quiere crear su propio compilador

Si tu objetivo es aprender a construir compiladores, este proyecto enseña varias lecciones muy concretas.

Primero: diseña por fases

No intentes empezar por “soportar todo C”.

Empieza por una tubería mínima y ve subiendo el nivel sólo cuando la base ya está estable.

Segundo: separa responsabilidad

No mezcles en el mismo sitio:

- parsing
- semántica
- IR
- backend
- encoding

Cada capa debe tener una misión clara.

Tercero: define un ABI temprano

Si no defines el contrato binario, el compilador crecerá con parches difíciles de mantener.

Cuarto: trata la memoria como una parte central del lenguaje

En PIC16 la memoria no es un detalle de implementación. Es parte del problema.

Quinto: mide y valida

Cuando el compilador empieza a tener ayuda runtime, páginas, bancos y perfiles de precisión, necesita:

- tests
- simulación
- reports de tamaño
- validación de layout

Sexto: sé honesto con los límites

El proyecto no promete:

- double
- ISO C completo
- tanf, atanf, powf, expf o logf
- NaN/Inf completos
- punteros de código genéricos

Y eso no es una debilidad. Es una decisión técnica madura.

12. Cómo leer el repositorio con criterio

Si quieres estudiar el proyecto como material de aprendizaje, este orden funciona bien:

1. README.md para entender el estado del proyecto y sus fases
2. DESIGN.md para entender la arquitectura general
3. docs/ir/overview.md para ver la representación intermedia
4. src/frontend/ para el análisis del lenguaje
5. src/ir/ para el lowering y las optimizaciones
6. src/backend/pic16/midrange14/ para la traducción a PIC16
7. docs/ir/ para seguir la historia fase por fase

Si te preguntas por qué ese orden funciona, la respuesta es simple: vas de lo conceptual a lo concreto, y no al revés.

13. Conclusión

pic16cc es un compilador pequeño, pero pedagógicamente muy rico.

No es interesante porque soporte todo. Es interesante porque obliga a resolver, de manera explícita, los problemas reales que todo compilador debe resolver:

- cómo se entiende el lenguaje
- cómo se valida
- cómo se representa internamente
- cómo se traduce a una máquina real
- cómo se mide el coste de esa traducción

- cómo se documentan sus límites

Si lees este proyecto con esa mentalidad, no sólo aprenderás cómo funciona `pic16cc`.

Aprenderás cómo pensar como alguien que diseña compiladores.

Glosario breve

ABI: contrato binario entre caller y callee.

AST: árbol de sintaxis abstracta.

Backend: parte que traduce la IR a la arquitectura destino.

CFG: grafo de flujo de control.

Fase: hito funcional del proyecto con un objetivo técnico concreto.

Frontend: parte que entiende el lenguaje fuente.

IR: representación intermedia.

ISA: conjunto de instrucciones de la CPU.

ISR: rutina de interrupción.

Lowering: traducción de una representación más abstracta a otra más concreta.

PCLATH: registro PIC16 usado para control de páginas en saltos y llamadas.

SFR: registro especial del hardware.

Stack-first: ABI en el que los argumentos se pasan por una pila de software.

W: acumulador principal PIC16.

Epílogo

Si quisieras empezar hoy tu propio compilador, el consejo más útil que deja `pic16cc` es este:

empieza pequeño, pero no empieces sin arquitectura.

Primero define la cadena completa. Después define el ABI. Luego añade memoria, control de flujo, tipos, helpers, validación y optimización. Y cuando la base ya funcione, amplía el lenguaje con la misma disciplina.

Ese orden no sólo produce un compilador. Produce un compilador que se puede entender, mantener y enseñar.